

Step 3 — A Triage Agent That Extracts a Structured Ticket From Free Text — Hints

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Only 2 hints — work through them in order, and don't jump ahead until you've genuinely tried. Each hint has 3 depth levels: **Basic** (the plain idea), **Intermediate** (proper Python), **Advanced** (what to do when the model's answer just doesn't fit, instead of only catching that it happened). Read Basic first even if you already know Python — it's the fastest way to spot exactly what each deeper level adds.

- [Hint 1 — Two separate jobs: shape, and survival](#)
- [Hint 2 — The plan, and almost the whole thing](#)

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 1 — Two separate jobs: shape, and survival

Basic Version

This step has two separate jobs: (1) get the model to hand back data that matches a fixed shape (`SupportTicket`) instead of free-form text, and (2) make the whole app survive the four common ways an LLM call can go wrong.

For the first part, you'll define a Pydantic model describing exactly what fields you want, and use the OpenAI SDK's structured-output feature so the model's reply is forced into that shape.

For the second part, think about each failure separately: a bad key, a rate limit, a conversation that's too long, and a reply that doesn't actually match your schema. Each one needs its own specific response, not one big catch-all.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

The schema, in `schemas.py` :

```
from typing import Literal
from pydantic import BaseModel

class SupportTicket(BaseModel):
    customer_name: str | None
    issue_category: Literal["billing", "technical", "account_access", "other"]
    urgency: Literal["low", "medium", "high"]
    summary: str
```

For extraction, use the SDK's parsing helper —

`client.beta.chat.completions.parse(model=..., messages=..., response_format=SupportTicket)` (check your installed SDK's exact method name against Doc04's material). The idea is the same either way: pass the Pydantic class itself as the expected shape, and get back an already-validated object at `.parsed`, not raw text you have to parse yourself. `customer_name: str | None` is what makes test input 2's "no name given" case work correctly — the field is allowed to be `None`, so the model has an honest way to say "not mentioned" instead of guessing one.

For the four failures, you're looking for these specific exception types (confirm exact names against your installed `openai` version): `AuthenticationError`, `RateLimitError`, a context-length error (often `BadRequestError`), and Pydantic's `ValidationError`. That's four `except` blocks, most specific first, not one big catch-all — each prints or logs something different, because a user typing a bad message needs a different response than your API key being wrong. Also test the key once, right when the app starts, instead of waiting for it to fail mid-chat — Step 1's Advanced hint covers exactly this.

Sketch `triage_extract(message: str) -> SupportTicket` and the four `except` blocks around it before writing the full function.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

Catching a `ValidationError` and giving up is the minimum bar. But not every validation failure means the same thing, and it's worth telling two very different cases apart before deciding what to do about one:

- **The model got zero info.** The message genuinely didn't contain enough to classify confidently — a one-word message, or something totally off-topic. The model shouldn't be guessing here, and a validation failure in this case is really the schema doing its job.
- **The model got it wrong.** The message had enough real information, but the model's answer just didn't fit your schema anyway — it returned a category that isn't one of your four `Literal` options, say, or left `summary` empty when there was clearly something to summarize. This is a formatting or judgment slip, not a lack of information.

`ValidationError` alone can't tell you which one happened — it only tells you the output didn't fit the shape. That's exactly why "catch it and fail" is a weak response: you're treating two different problems identically.

A genuinely better response is to retry **once**, with a clearer instruction that explains what went wrong the first time. For example, spell out the exact allowed values for `issue_category` and `urgency` again — a model that almost got it right often just needs the rule stated more firmly. Only one retry, not a loop: if the second attempt also fails, give up and fail clearly. Retrying forever on a message the model genuinely can't classify wastes calls and won't fix a truly ambiguous input — that's a job for a human, not another API call.

Difference between Basic, Intermediate, and Advanced: Basic and Intermediate catch a validation failure and stop there — correct, but final. Advanced treats the *first* failure as a chance to give the model one more, better-informed try. It also separates "genuinely no information" from "had the information, got the format wrong" as two different reasons a retry might or might not help.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Hint 2 — The plan, and almost the whole thing

Basic Version

The plan, in plain steps:

```
define SupportTicket with: customer_name (can be empty), issue_category, urgency,

function triage_extract(message):
    try:
        ask the model to extract a SupportTicket from this message
        return the parsed ticket
    if the key is wrong: say so clearly
    if we hit a rate limit: say so clearly
    if the conversation got too long: say so clearly
    if the model's answer doesn't match SupportTicket: say so clearly

test:
    run all 3 example inputs through it
    check input 2 really gives customer_name = None
```

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Intermediate Version

The same plan, closer to real structure:

```
schemas.py:
class SupportTicket(BaseModel):
    customer_name: str | None
    issue_category: Literal["billing", "technical", "account_access", "other"]
    urgency: Literal["low", "medium", "high"]
    summary: str

chat_client.py (added):
def check_api_key_at_startup(client: OpenAI) -> None:
    try:
        client.chat.completions.create(model="gpt-4o-mini",
            messages=[{"role": "user", "content": "ping"}], max_tokens=1)
    except AuthenticationError:
        raise SystemExit("Your OPENAI_API_KEY is invalid. Fix your .env before")

def triage_extract(client: OpenAI, message: str) -> SupportTicket:
    response = client.beta.chat.completions.parse(
        model="gpt-4o-mini",
        messages=[
            {"role": "system", "content": "Extract a support ticket. Leave customer information blank if not present."},
            {"role": "user", "content": message},
        ],
        response_format=SupportTicket,
    )
    return response.choices[0].message.parsed

main.py, /extract mode:
try:
    ticket = triage_extract(client, user_text)
    print(ticket)
except AuthenticationError: ...
except RateLimitError: ...
except BadRequestError: ...      # covers the too-long-conversation case
except ValidationError: ...

test_data.py:
the 3 example inputs, saved once, reused for every test run.
```

Write the four `except` blocks (bodies can log and print a clear per-case message) and run all 3 test inputs, specifically confirming input 2 gives `customer_name=None` , before moving to Advanced.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Advanced Version

Here's the retry-once shape — try finishing the second `except` block yourself before checking the Solution:

```
def triage_extract(client: OpenAI, message: str, extra_instruction: str = "") ->
    system_content = "Extract a support ticket. Leave customer_name empty if not
    if extra_instruction:
        system_content += " " + extra_instruction
    response = client.beta.chat.completions.parse(
        model="gpt-4o-mini",
        messages=[
            {"role": "system", "content": system_content},
            {"role": "user", "content": message},
        ],
        response_format=SupportTicket,
    )
    return response.choices[0].message.parsed

def triage_extract_with_retry(client: OpenAI, message: str) -> SupportTicket:
    try:
        return triage_extract(client, message)
    except ValidationError as first_error:
        logger.warning("First extraction attempt failed validation: %s", first_error)
        clarification = (
            "Your previous answer didn't match the required ticket format. "
            "issue_category must be exactly one of: billing, technical, account_a
            "urgency must be exactly one of: low, medium, high. Try again, carefu
        )
        # your turn: call triage_extract() again with this clarification, inside
        # try/except ValidationError – if it fails a second time, log it and re-r
```

Test this against a message deliberately worded to be borderline (short, vague, or off-topic) and watch whether the retry actually helps or whether it fails cleanly a second time — both are correct outcomes, depending on the input.

Difference between Basic, Intermediate, and Advanced: Basic and Intermediate catch `ValidationError` and stop. Advanced gives the model one more chance, with a more specific instruction about exactly what it got wrong — and only one chance, so a truly unclassifiable message still fails cleanly instead of looping.

[Back to this step](#) · [Hint 1](#) · [Hint 2](#) · [Solution](#)

Full solution: [Show me the solution](#)